

Query Store Triage and Stored Procedure Refactor Program

Author: Justin Hunter

Date: 2026-05-08

Audience: Engineering team and engineering leadership

Companion documents: Program Charter, Maintenance Operations Plan, Scalability Roadmap

1. Purpose

This document describes the active stored procedure refactor program: the methodology the MFC DBA team and I are applying, the diagnostic toolkit that feeds it, the procedure-by-procedure status (including current deployment state), and the expected outcomes at the program level. It sits underneath the program charter and goes deeper on the engineering work.

The shorter framing is straightforward. The PharmAssist database has a relatively small number of procedures and ad hoc queries that are doing a disproportionate share of the I/O across the fleet. Refactoring those procedures and addressing a small number of supporting index gaps will deliver the bulk of the near-term performance relief that the program needs. Each refactor also produces a piece of durable knowledge that future engineers can apply to procedures we have not touched yet.

The program runs on two clocks. The analysis clock is the rate at which the MFC DBA team and I produce deployable recommendations with complete evidence packages. The deployment clock is the rate at which the iA DBA team and the iA software development teams move those recommendations through review, validation, non-production testing, and production deployment. The two clocks run at different speeds. The program charter describes the working relationship between the teams. This document describes both clocks and reports against both. The work product is a documented, evidence-backed backlog of recommendations, organized so that the iA team can prioritize and deploy at their cadence.

2. Evidence Base

The starting point for every triage decision is the cross-MFC Query Store capture from May 7, 2026: the top fifty offenders per site, on a thirty-day lookback, one tab per MFC. Fourteen of

fifteen sites are reporting in the May 7 capture. The missing site (Indy) is not in the top fifty for any procedure we have triaged so far, and its absence is itself a signal worth investigating.

The capture supports several views.

The first is the procedure level roll-up. For each procedure, I aggregate executions, total reads, average reads per execution, and total CPU across sites. This view ranks procedures by their fleet-wide impact and is what feeds the triage prioritization in the tracking sheet.

The second is the plan stability view. For each procedure at each site, I count the distinct plan variants captured for the same query identifier. A statement that appears multiple times at the same site under different plan variants is producing different plans on different compilations, which is the textbook signature of parameter sniffing or insufficient index coverage. Plan stability matters because the worst case plan can be orders of magnitude more expensive than the best case plan.

The third is the per-site spread. For the same procedure across sites, the spread in reads per execution can range from trivial to dramatic. `lsp_ShpGetOrdersForTopReadyToShipGroup` shows a 37x spread between the lowest cost site (Kent at 1,685 reads per execution) and the highest cost site (NorthLake at 63,175). `lsp_PmssIwebGetTopQueuedTrx` shows a 70,000x spread (West Jordan at 20 reads, Orlando at 1.4 million). Spread of this magnitude points to data shape variance and to plans that are not stable across the shape variance.

These three views are what I look at first on every procedure. Together they answer the questions that any refactor decision has to answer: how big is the problem, where is it worst, and what is the proximate cause.

3. Diagnostic Toolkit

Underneath the cross-MFC capture sits the diagnostic toolkit: a deployable set of ten numbered scripts that profiles a site in roughly ninety seconds. Engineers and operations staff can run the kit against any site without my involvement.

The scripts in order are:

1. **Query Store top offenders.** Identifies the procedures and statements consuming the most reads in the configured lookback window.
2. **Plan instability detector.** Surfaces statements with high read ratio variance across plan variants, which is the parameter sniffing canary.
3. **Forced plan failure audit.** Confirms that any forced plans are still binding, since silent forced plan failures are a common cause of mysterious regressions.

4. **Wait stats snapshot.** Server level wait stats so I can tell the difference between an I/O bound system, a lock bound system, and a CPU bound system.
5. **Index usage and missing indexes.** Identifies unused indexes that are paying maintenance cost for nothing and missing index recommendations from the optimizer.
6. **Ship tote skew analysis.** A procedure-specific skew analysis script for the `SrtShipToteShipmentAssoc` table.
7. **Statistics staleness.** Surfaces tables whose statistics are stale enough to risk bad plan choices.
8. **TempDB and memory pressure.** Catches resource pressure that masquerades as query level slowness.
9. **Generic skew analysis.** A reusable parameterized skew analysis for any table and column pair.
10. **Data velocity and capacity.** Per-table growth rates, used for capacity planning and partitioning retention decisions.

The toolkit is intentionally read only. Every script is safe to run on production and uses NOLOCK or DMV reads.

4. Workflow

The refactor workflow is documented in detail elsewhere in our analysis library. The summary here captures the structure.

For every procedure, we produce a self-contained analysis package containing three artifacts: a snapshot of the live DDL at the start of analysis, the deployment-ready refactored body with inline comments at every changed block, and a written analysis that follows an eleven-section template:

1. **Procedure name and surface area.** The single sentence of what the procedure does, the tables it touches, the indexes it uses, the callers if known.
2. **Overview of performance.** A few short paragraphs setting the cost picture from the cross-MFC view.
3. **Evidence of original.** The cross-MFC Query Store table and the `STATISTICS IO`, `STATISTICS TIME` output from a representative site.
4. **Issue identification.** The specific anti-patterns and the line numbers where they live.
5. **First principles.** One short subsection per principle with citations to the relevant entry in the masterclass library. Net new principles trigger a new entry.
6. **Refactor (commented).** The new SQL with inline comments explaining each changed block.

7. **Risk and rollback.** What could go wrong, what to watch for in the first twenty four hours, how to revert.
8. **Evidence of refactor.** STATISTICS IO , STATISTICS TIME from the warm cache run on the same data state as section three.
9. **Comparison and improvement.** The apples-to-apples table comparing original and refactor on reads, CPU, elapsed, and plan shape. Verdict at the top of the section.
10. **Validation checklist.** Marked pass or fail against seven explicit criteria. Net call sentence at the end.
11. **Open items and future improvements.** Index recommendations, follow-on procedure changes, and schema level changes that need DBA blessing.

The eleven sections are not optional. Even small procedures get the full template, because the consistency of the form is more valuable than the brevity of any one document. Engineers and leadership should be able to read six analyses in a sitting without recalibrating per procedure.

The apples-to-apples evidence protocol that gates section nine and section ten is the most important rule in the workflow. Same data state, warm cache, non-zero result set, plan shape matches the prediction, identical row count and identities. Any rule violation invalidates the comparison. The rule is in writing because I have learned the hard way that promising looking warm cache results turn out to be data-state luck more often than I would like.

5. Procedure-by-Procedure Status

The triage prioritization is the MFC database optimization tracking sheet. The active deep-dive cohort starts at Row 15 and runs through Row 39. Earlier rows have already been worked through individually, including the two procedures that have moved furthest through the deployment pipeline to date (`lsp_SrtGetShipToteIfExists` , in production, and `lsp_RxfGetListOfManualFillGroups` , currently in iA review).

The status table below tracks two states per procedure: the analysis state (where the recommendation sits in our cataloging workflow) and the deployment state (where the recommendation sits in the iA review and deployment pipeline). The deployment state values are: Cataloged (recommendation published, not yet handed off), In iA Review (with the iA DBA team or with iA developers for validation), In Non-Prod Testing, In Production, and Deferred.

Row	Procedure	Priority	Analysis State	Depl State
Pre-15	<code>lsp_SrtGetShipToteIfExists</code>	(earlier cohort)	Complete	In Pro

Row	Procedure	Priority	Analysis State	Depl State
Pre-15	lsp_RxfGetListOfManualFillGroups	(earlier cohort)	Complete	In iA
15	lsp_ShpGetOrdersForTopReadyToShipGroup	P3	Pilot complete; awaiting STATS captures	Catal
16	lsp_SrtGetSortationItems	P2	Analysis drafted, refactor staged	Catal
17	lsp_SrtGetNumOfStoresWithAndWithoutSorterLocs	P1	Analysis drafted, refactor staged	Catal
18	lsp_OrdGetRxStatusCounts	P3	Analysis drafted, refactor staged	Catal
19	lsp_RxvGetRxsQueuedForVerification	P3	Analysis drafted, refactor staged	Catal
20	lsp_PmssIwebGetTopQueuedTrx	P3	Analysis drafted; verdict is index addition, not body change	Catal
21	lsp_RbtDetermineNextVialToProcess	P3	Analysis drafted, refactor staged	Catal
22-39	Cross-list expense candidates added 2026-04-27	mixed P0/P1/P2	Queued	Catal (penc

Rows 15 through 21 each have their own written analysis and are the right place to read for procedure-level detail. The fleet-wide load these procedures carry today is summarized below.

Procedure	Executions / month (fleet)	Total reads / month (fleet)	Avg reads / exec
<code>lsp_ShpGetOrdersForTopReadyToShipGroup</code>	33.1M	950B	28,659
<code>lsp_PmssIwebGetTopQueuedTrx</code>	825M	278T	337,000
<code>lsp_RbtDetermineNextVialToProcess</code>	43.4M	705B	16,250
<code>lsp_SrtGetSortationItems</code>	(see Analysis)	(see Analysis)	(see Analysis)
<code>lsp_SrtGetNumOfStoresWithAndWithoutSorterLocs</code>	(see Analysis)	(see Analysis)	(see Analysis)
<code>lsp_OrdGetRxStatusCounts</code>	(see Analysis)	(see Analysis)	(see Analysis)
<code>lsp_RxvGetRxsQueuedForVerification</code>	(see Analysis)	(see Analysis)	(see Analysis)

The procedure-level evidence and refactor commentary live in the per-procedure analysis packages. Anyone reading this document who wants depth should ask for the relevant package by procedure name.

5.1 Deployment Pipeline and Handoff

Once an analysis passes our internal validation checklist, it is published as a recommendation to the iA DBA team. The handoff package contains the snapshot of the original DDL, the refactored body with inline commentary, the written analysis, the saved execution plans, and the STATS IO and STATS TIME captures. The intent is that the iA reviewer can read the package and reach the same conclusion we did, with the evidence already in front of them.

After the iA DBA team's review, the recommendation moves to the iA developers responsible for the affected functional area for validation. They confirm that the change preserves application contract, that there are no behavioral side effects in calling code paths, and that the refactor is consistent with how that area of the codebase is intended to evolve. After the iA developer review, the change moves into non-production testing. The duration of the test window varies by risk profile and by iA team bandwidth. Production deployment follows a successful test window.

As of this document, the deployment pipeline has produced one production deployment (`lsp_SrtGetShipToteIfExists`) and one in-review item (`lsp_RxfGetListOfManualFillGroups`) in the seven-week window since the deep-dive program began producing output. The cataloged backlog from the cohort and from prior work is materially larger than that. The procedures in the cataloged backlog are the same procedures that have been generating cancellations and stored-procedure performance failures in production over the same window. The mismatch between the analytical pace and the deployment pace is therefore not a hypothetical concern.

The mitigation on our side is to make every recommendation as easy as possible to accept. The analysis template carries the risk and rollback section, the validation checklist, and the explicit verdict so that the iA reviewer is not asked to derive any of those from scratch. The masterclass entries are referenced by name so that the underlying principles are available to the reviewer in plain prose. Where an information request goes from us to the iA team, we record it against the relevant analysis and proceed with a defensible assumption rather than blocking on the request.

The shared mitigation, raised as a conversation rather than as an action item in this document, is whether the multi-stage review can be tightened in cases where the recommendation is small, low risk, and carries a complete evidence package. That conversation belongs at the program level rather than on a per-procedure basis.

5.2 Post-Deployment Monitoring and Per-MFC Variance Tracking

A refactor deployed to production is not a refactor closed. The MFC DBA team and I monitor every deployed procedure at every site for a defined window after rollout and compare the actual behavior against the pre-deployment baseline at the same site. This step is part of the workflow, not an extra. Without it, we have no evidence that the deployed code is producing the outcome the analysis predicted.

The reason post-deployment monitoring is non-optional is that the cross-MFC Query Store evidence already shows wide per-site variance on the same statement. The Row 15 procedure varies by 37x in reads per execution between its lowest-cost site and its highest-cost site. The Row 20 statement varies by roughly 70,000x. That variance exists today, before any refactor work. It does not disappear when a refactored body is deployed. A refactor optimized against the median data shape can produce a meaningful win at the median sites, no measurable

change at sites where the bottleneck was somewhere else, and a measurable regression at sites whose data shape interacts badly with the refactored plan. All three outcomes are possible from a single uniform code change. Without monitoring, we will not know which outcome each site actually got.

The monitoring window per procedure is two full weeks of normal production traffic at the site, measured against a comparable two-week window from before the deployment. Two weeks is long enough to span the natural variation in workload (weekends, end-of-month, batch peaks) and short enough to surface a regression before it accumulates into an operational event. The metrics captured per site are total reads, average reads per execution, plan variants observed, average duration, and the distribution of reads per execution across executions. These are the same metrics the cross-MFC capture already produces, so the monitoring is mechanical: pull the post-deployment slice, pull the pre-deployment slice, compare.

Each post-deployment captured run lands in a per-procedure post-deployment record with one row per site. The record makes the per-MFC outcome explicit:

- **Improvement consistent with prediction.** The site shows the read reduction the analysis predicted, within the expected variance band. The change is closed at that site.
- **No measurable change.** The site shows neither improvement nor degradation. The dominant cost at that site was somewhere other than what the refactor addressed. We document that finding and consider whether a follow-on analysis is warranted, but the refactor itself is closed at that site.
- **Degradation against baseline.** The site shows higher reads, longer duration, or new plan variants compared to the pre-deployment baseline. The refactor is not closed at that site. A per-site root cause analysis is opened.

The per-site root cause analysis follows a simple template. We pull the new execution plan at the affected site and compare it to the predicted plan from the analysis. We check whether the index choice matches the prediction. We check whether the data distribution at the site matches the assumptions the refactor was built against (this is where access-pattern variance shows up). We check whether parameter sniffing on the first compilation at the site landed on a non-representative parameter set. We check whether statistics on the affected tables are stale at that site. The output of the root cause analysis is a written explanation that lands in the lessons-learned log and is shared with the iA team. The explanation drives one of three resolutions:

- **An explanation that grounds the variance in data shape.** The variance is real and is attributable to the data the site is operating against. The refactor is not at fault. The site is closed with the explanation on file.
- **A refinement to the deployed code.** A hint, an `OPTION (RECOMPILE)`, an `OPTIMIZE FOR`, a parameter pre-assignment, or a small data-aware branch that addresses the outlier

without breaking the median. The refinement is published as a follow-up recommendation through the same review pipeline.

- **An accepted exception at the site.** The site is operating outside the envelope the refactor was designed for. We acknowledge the variance, document it, and treat the site as a known exception until either the data shape converges or a more substantial change is warranted.

The methodology does not prescribe which of the three resolutions is right in any given case. That decision is shared with the iA team because it interacts with the iA team's preference for codebase uniformity. What the methodology does insist on is that the variance is investigated rather than ignored, that the explanation is written down rather than left implicit, and that the resolution is captured against the procedure's record rather than re-derived later.

The honest framing is this: uniform code interacting with non-uniform data cannot be assumed to produce uniform outcomes. We can hold that line on the input side (one body of code at every site) only if we are willing to accept variance on the output side (different procedures producing different results at different sites) or if we are willing to add minor data-aware accommodations to the body that allow it to behave well across the variance. Pretending the variance does not exist is the only option that is not viable, because the data already shows it.

6. First Principles

The refactors apply a recurring set of patterns. The patterns are documented in the masterclass library as plain prose notes. The notes are intended to be readable in a year by someone who does not have the context of the original refactor.

The current entries in the library cover:

- Correlated subqueries to CTEs
- FOR XML PATH consolidation
- LEFT JOIN OR anti-pattern
- Parameter sniffing
- Non-SARGable predicates
- Density vector
- Table variables vs temp tables
- Scalar UDF parallelism barrier
- NOLOCK strategy
- TOP with ORDER BY semantics
- FORCESEEK hints
- Query Store triage

- UNION ALL views
- Index key columns vs included columns
- Catch-all query anti-pattern
- NOT IN vs NOT EXISTS
- Conditional aggregation consolidation
- STRING_SPLIT vs WHILE loop CSV parsing
- Ambiguous self-comparison predicates
- A SQL Server performance overview that ties the rest together

Net new entries are written when a refactor surfaces a principle that no existing entry covers. The Row 15 pilot produced the FOR XML PATH consolidation entry because the pattern (the same FOR XML PATH expression repeated identically across multiple UNION branches, consolidate the driver set first and run the expression once over the union) was not previously named.

A common anti-pattern to fix mapping is part of the workflow document. It pairs each anti-pattern with the recommended fix and a citation to the relevant masterclass entry. Engineers can use the table to triage a procedure they have not seen before. Anti-patterns covered include parameter sniffing, correlated scalar subqueries, scalar UDFs, LEFT JOIN with OR across different join paths, table variables joined to large tables, non-SARGable predicates, catch-all WHERE clauses, TRIM on column in WHERE, NOT IN with NULL risk, scalar `(Select 1 ...) = 1` checks, UNION when label columns prevent overlap, and legacy temp-table existence checks.

7. Expected Outcomes

I expect the refactor program to deliver three categories of improvement, each of which is realized only after the corresponding recommendation is deployed to production.

The first is direct read reduction at the procedure level. Each refactored procedure is targeted at a specific reduction range supported by the analysis. The aggregate target across the deep-dive cohort is a 25 to 40 percent reduction in fleet-wide reads for the procedures in the cohort, measured against the May 7, 2026 baseline once the cohort has been deployed. Some procedures will deliver more, some less, and the analyses will say so plainly. The procedure where the win is an index, not a code change (`lsp_PmssIwebGetTopQueuedTrx`), will deliver the largest single win in absolute terms.

The second is plan stability. Several of the procedures in the cohort show multiple plan variants for the same statement at the same site, and several show extreme reads-per-execution

variance across sites. Stable plans are an outcome in their own right because they translate into predictable response time at the application layer, not just lower average cost.

The third is reduced operational surprise. Plans that flip under load surface as performance incidents. Procedures that scale linearly with row count in a table that is growing exponentially eventually surface as outages. Refactoring the procedures and tightening the indexes reduces the rate at which the database produces these surprises. The cancellations and stored-procedure performance failures that have been observed against procedures already on the optimization list are the live evidence that this category of outcome matters in operational terms, not only as a metric.

There is a fourth outcome that is harder to measure but matters for the longer arc. The masterclass library, the lessons-learned log, and the per-procedure analyses are knowledge artifacts. Any engineer joining either the MFC DBA team or the iA team can read them and bring themselves up to the same standard of work without my involvement. By the end of Phase 3 of the program, the working knowledge of how to do this kind of optimization is shared across both teams.

A fifth outcome, particular to the current pace dynamic, is the cataloged backlog itself. Even where production deployment lags the analysis, the backlog is durable value: it is a documented, evidence-backed inventory of the largest performance levers available in the database, organized so that future engineering capacity (whether on the iA side or by adding capacity to the MFC DBA team) can act against it without redoing the analysis. The backlog persists; the deployment cadence is what determines how quickly it converts.

8. Validation and Sign-Off

Validation runs in two stages. The pre-deployment stage gates whether a recommendation ships. The post-deployment stage gates whether a deployed refactor is declared closed at each site.

8.1 Pre-Deployment Validation

A refactor is not handed to the iA team for review until it has passed the pre-deployment validation checklist in section ten of its analysis. The checklist is reproduced here for reference:

- Same data state. Captures taken back to back, ideally with a data freeze.
- Warm cache only. Two runs, cold cache numbers discarded.
- Non-zero result set. Both runs returned at least one row.
- Identical result set. Same row count and same row identities for deterministic queries.

- Plan shape matches prediction. The optimizer used the indexes we expected. Confirmed against the saved execution plan file.
- No new error or warning messages. No cardinality errors, conversion warnings, or excessive memory grant warnings.
- Warm-cache elapsed time at or below original. Wall clock has not regressed.

A refactor that fails any of these criteria does not ship. If the failure is due to a data state issue, the comparison is rerun. If the failure is due to a missing prediction (the optimizer chose a different index than we predicted), the analysis is revisited and either the refactor changes or the prediction was wrong and we explain why and adjust.

8.2 Post-Deployment Validation

A refactor that has been deployed to production is closed at a site only when the post-deployment validation against that site has been completed. The post-deployment checklist, applied per site, is:

- A two-week post-deployment window has been captured against the same Query Store views used for the pre-deployment baseline.
- The procedure's actual reads, duration, and plan variants are documented for the post-deployment window at that site.
- The actual outcome is compared against the prediction the analysis made for that site (or for the median site, where the analysis did not predict per-site outcomes).
- The site is recorded as Improved, Unchanged, or Regressed against its own pre-deployment baseline.
- For any site recorded as Regressed, a per-site root cause analysis has been opened, run, and documented in the lessons-learned log.

A site is closed only when its row in the post-deployment record carries a written outcome and, where the outcome is regression, a written resolution path (explanation, refinement, or accepted exception). A refactor is closed at the program level only when every site has been closed at the per-site level. That is what "deployed and validated" means in this program.

9. Cadence and Reporting

I report against the program weekly during a standing review. The review consists of a status walk-through against the tracking sheet (both analysis state and deployment state), a sign-off on completed analyses, and a working session on whichever procedure is currently in flight. Quarterly, I publish a refreshed cross-MFC Query Store capture and a delta-against-baseline report at the program level so that leadership can see the aggregate effect.

The reporting carries three distinct numbers. The first is the analytical pace: the count of analyses completed and published as deployable recommendations per quarter, with each one carrying a complete evidence package. The second is the deployment pace: the count of recommendations that have moved from cataloged through production over the same window. The third is the post-deployment outcome distribution across sites for every refactor that has reached production: the count of sites closed as Improved, the count closed as Unchanged, and the count still open as Regressed. Reporting all three numbers, side by side, gives leadership and the iA team a shared view of where the program is producing realized value and where there is open work. The cataloged-versus-deployed gap is currently large. The deployed-versus-closed gap will become visible as more refactors reach production. Making both gaps visible is the first step in shrinking them.

Between review cycles, the analyses themselves are the running record. Each one is timestamped and dated against the data state it was captured on. Anyone who wants to understand the state of the program at a point in time can read the relevant analysis and arrive at the same conclusions we would.

10. Dependencies on the Other Streams

The triage program is not strictly dependent on the maintenance plan or the scalability roadmap, but it interacts with both.

It interacts with the maintenance plan in two places. First, several of the procedures in the deep dive cohort touch tables that are also subject to nightly purge. Coordinating refactor deployments with the next maintenance window reduces the risk of a refactor regression overlapping a purge regression. Second, a few of the index recommendations in section eleven of the analyses imply maintenance changes (statistics rebuild cadence, fragmentation thresholds for the new indexes), and those changes are folded into the maintenance plan rather than into the refactor recommendation.

It interacts with the scalability roadmap in one place. Several of the procedures in the deep dive cohort access the high-volume tables that are partitioning candidates. Once those tables are partitioned, the procedures benefit from partition elimination automatically as long as their predicates are partition-aligned. Some of the analyses already note this. The section eleven open items point at the partitioning effort where relevant.

The longer story on the architectural changes that surfaced during the refactor work (Service Broker for the PMSS polling pattern, partitioned table support for the highest volume statement)

is in the Scalability Roadmap. They are too large in scope for the per-procedure analyses to carry alone.